```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

from sklearn.datasets import make_moons,load_iris
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans,DBSCAN,AgglomerativeClustering
from sklearn.neighbors import NearestNeighbors
from scipy.cluster.hierarchy import dendrogram,linkage
```

Individual Household Electric Power Consumption Dataset

- Measurements of electric power consumption in one household with a one-minute sampling rate over a period of almost 4 years. Different electrical quantities and some sub-metering values are available.
- Number of features in the dataset : **9**
- Number of instances : **2075259**
- FEATURES :
  - date: Date in format dd/mm/yyyy
  - time: time in format hh:mm:ss
  - global_active_power: household global minute-averaged active power (in kilowatt)
  - global_reactive_power: household global minute-averaged reactive power - (in kilowatt)
  - voltage: minute-averaged voltage (in volt)
  - global_intensity: household global minute-averaged current intensity (in ampere)
  - sub_metering_1: energy sub-metering No. 1 (in watt-hour of active energy). It corresponds to the kitchen, containing mainly a dishwasher, an oven and a microwave (hot plates are not electric but gas powered).
  - sub_metering_2: energy sub-metering No. 2 (in watt-hour of active energy). It corresponds to the laundry room, containing a washing-machine, a tumble-drier, a refrigerator and a light.
  - sub_metering_3: energy sub-metering No. 3 (in watt-hour of active energy). It corresponds to an electric water-heater and an air-conditioner.

Reading the dataset

```
!pip install ucimlrepo
```

```
Collecting ucimlrepo
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (20
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucim
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->uciml
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->uci
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
Installing collected packages: ucimlrepo
Successfully installed ucimlrepo-0.0.7
```

```
from ucimlrepo import fetch_ucirepo

individual_household_electric_power_consumption = fetch_ucirepo(id=235)
X = individual_household_electric_power_consumption.data.features
```

```
/usr/local/lib/python3.12/dist-packages/ucimlrepo/fetch.py:97: DtypeWarning: Columns (2,3,4,5,6,7) have mixed typ
  df = pd.read_csv(data_url)
```

```
# print(individual_household_electric_power_consumption.metadata)

# print(individual_household_electric_power_consumption.variables)
```

Preview of the dataset

```
X.head()
```

	Date	Time	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_me
0	16/12/2006	17:24:00	4.216	0.418	234.840	18.400	0.000	
1	16/12/2006	17:25:00	5.360	0.436	233.630	23.000	0.000	
2	16/12/2006	17:26:00	5.374	0.498	233.290	23.000	0.000	
3	16/12/2006	17:27:00	5.388	0.502	233.740	23.000	0.000	
4	16/12/2006	17:28:00	3.666	0.528	235.680	15.800	0.000	

Preprocessing

Reducing the number of features by combining date and time features and converting the day-time format to seconds since 1970

```
X['Date'] = X['Date'] + ' ' +X['Time']
X['Date'] = pd.to_datetime(X['Date'],dayfirst=True) - pd.Timestamp("1970-01-01")
X['Date'] = X['Date'].dt.total_seconds()
X = X.drop('Time',axis="columns")
X.head()
```

	Date	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering
0	1.166290e+09	4.216	0.418	234.840	18.400	0.000	1.0
1	1.166290e+09	5.360	0.436	233.630	23.000	0.000	1.0
2	1.166290e+09	5.374	0.498	233.290	23.000	0.000	2.0
3	1.166290e+09	5.388	0.502	233.740	23.000	0.000	1.0
4	1.166290e+09	3.666	0.528	235.680	15.800	0.000	1.0

Replacing ? with nan

```
for col in X.columns[1:-1]:
  X[col] = X[col].replace("?",np.nan)
```

Changing the datatype of the feature from string to float for better processing

```
non_float_cols = X.columns[1:-1]
for col in non_float_cols:
  X[col] = X[col].astype(float)
```

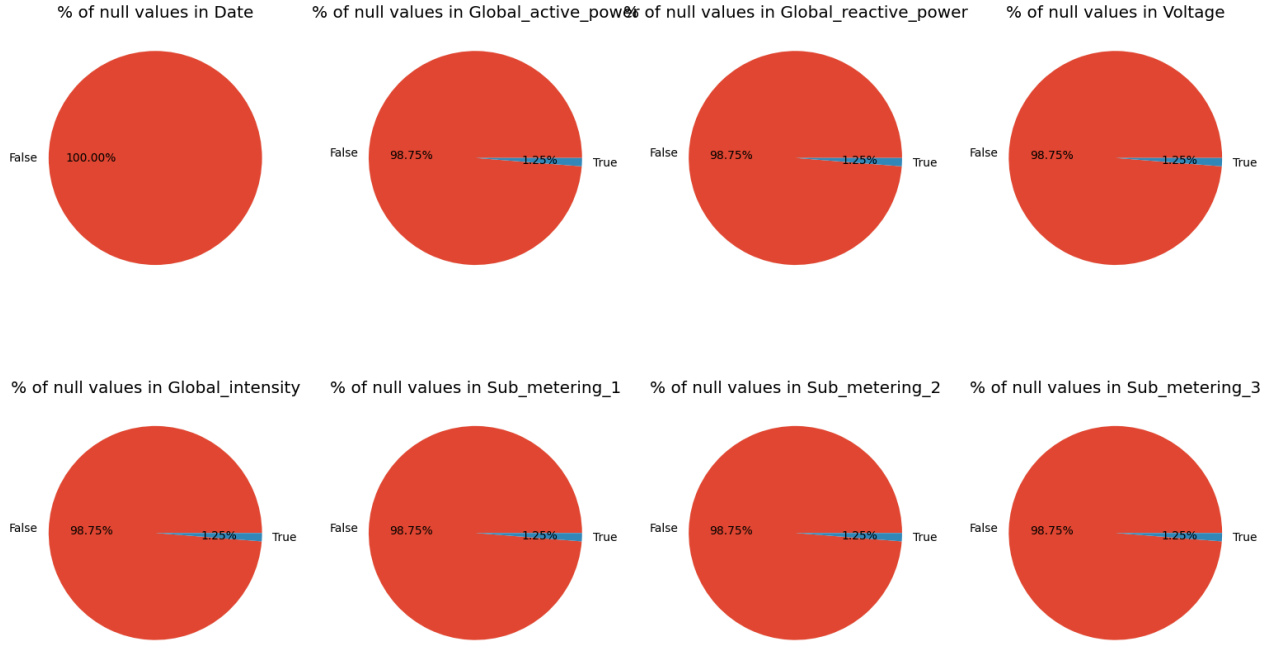
Dealing with nan values

```
X.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2075259 entries, 0 to 2075258
Data columns (total 8 columns):
 #   Column                 Non-Null Count    Dtype
---  ------                 --------------    -----
 0   Date                   2075259 non-null  float64
 1   Global_active_power    2049280 non-null  float64
 2   Global_reactive_power  2049280 non-null  float64
 3   Voltage                2049280 non-null  float64
 4   Global_intensity       2049280 non-null  float64
 5   Sub_metering_1         2049280 non-null  float64
 6   Sub_metering_2         2049280 non-null  float64
 7   Sub_metering_3         2049280 non-null  float64
dtypes: float64(8)
memory usage: 126.7 MB
```

```
plt.style.use("ggplot")
plt.figure(figsize=(19.2,10.8))
for col in X.columns:
  na_dist = X[col].isna().value_counts(normalize=True)
```

```
  index = X.columns.get_loc(col) + 1
  plt.subplot(2,4,index)
  plt.title(f"% of null values in {col}")
  plt.pie(na_dist,labels=na_dist.index,autopct='%.2f%%')
plt.show()
```




```
imputer = SimpleImputer(strategy='mean')
for col in X.columns[1:]:
  X[col] = imputer.fit_transform(pd.DataFrame(X[col]))
X.head()
```

	Date	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering
0	1.166290e+09	4.216	0.418	234.84	18.4	0.0	
1	1.166290e+09	5.360	0.436	233.63	23.0	0.0	
2	1.166290e+09	5.374	0.498	233.29	23.0	0.0	
3	1.166290e+09	5.388	0.502	233.74	23.0	0.0	
4	1.166290e+09	3.666	0.528	235.68	15.8	0.0	

Normalization

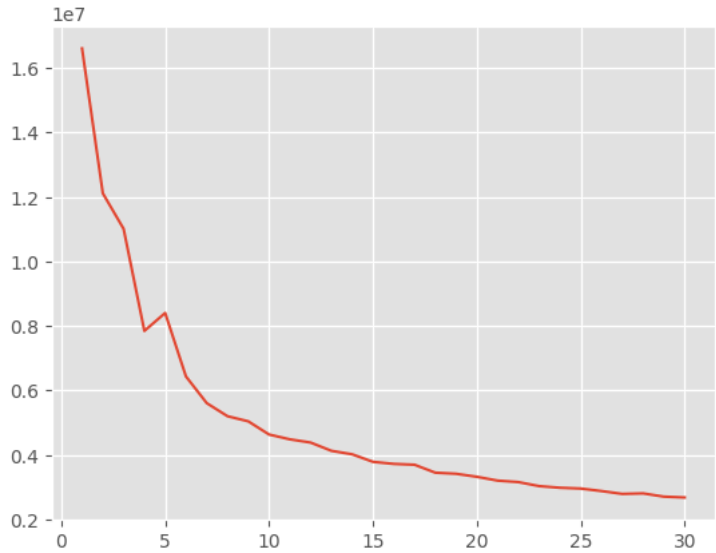
```
norm = StandardScaler()
X_norm = norm.fit_transform(X)
X = pd.DataFrame(X_norm,columns=X.columns)
X.head()
```

	Date	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2
0	-1.732050	2.973749	2.627217	-1.863517	3.118369	-0.183489	-0.051598
1	-1.732048	4.062593	2.787911	-2.239335	4.159920	-0.183489	-0.051598
2	-1.732047	4.075918	3.341412	-2.344937	4.159920	-0.183489	0.121249
3	-1.732045	4.089243	3.377122	-2.205170	4.159920	-0.183489	-0.051598
4	-1.732043	2.450266	3.609235	-1.602618	2.529666	-0.183489	-0.051598

KMeans clustering

Finding the optimal value of K by using elbow method

```
inertia = dict()
for k in range(30):
  km = KMeans(n_clusters=k+1)
  km.fit(X)
  inertia[k+1] = km.inertia_
plt.style.use("ggplot")
sns.lineplot(inertia)
plt.show()
```



Applying Kmeans to the dataset with the optimal K value obtained : **22**

```
km_optimal = KMeans(22)
fit = km_optimal.fit(X)
centers = fit.cluster_centers_
```

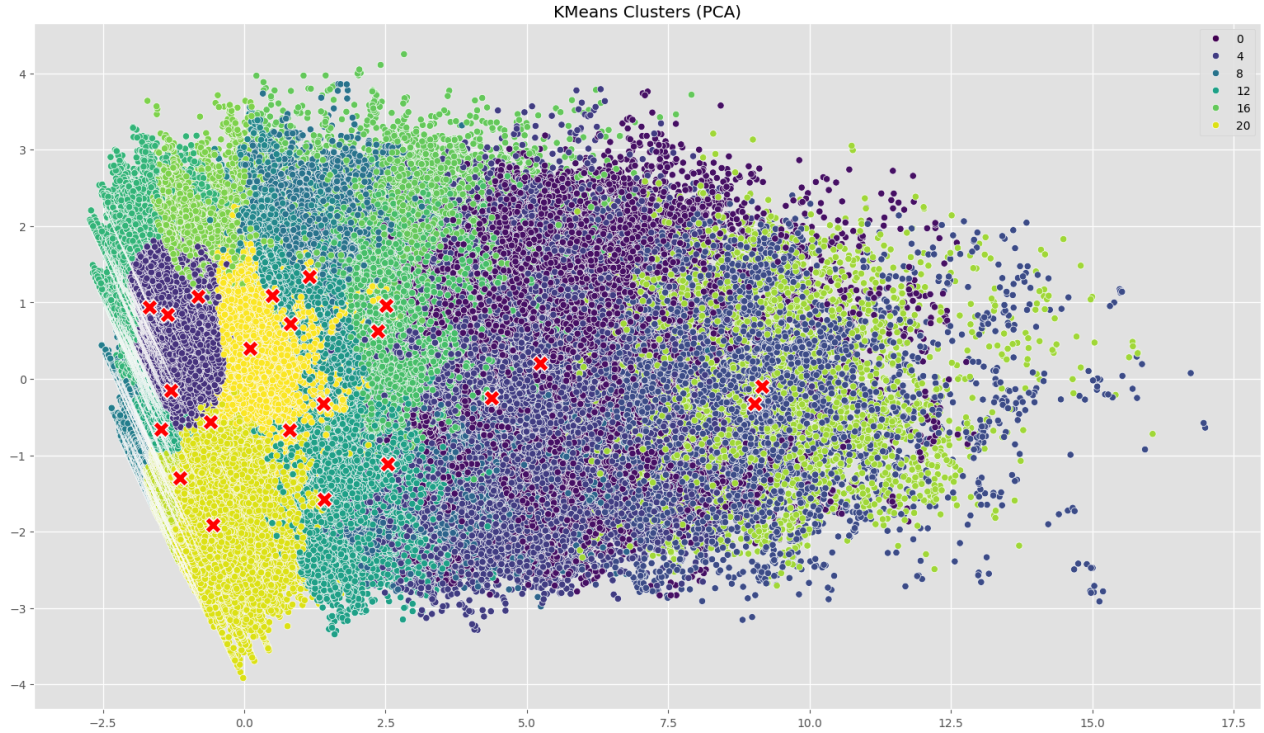
Visualising the clusters obtained through KMEANS

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

centers_pca = pca.transform(centers)
plt.figure(figsize=(19.2,10.8))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=fit.labels_, palette='viridis')
sns.scatterplot(x=centers_pca[:,0], y=centers_pca[:,1], color='red', marker='X', s=200)

plt.title("KMeans Clusters (PCA)")
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feat
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Creating legend with loc="be
  fig.canvas.print_figure(bytes_io, **kw)
```



Agglomerative Heirarchical Clustering

Iris dataset

- Iris dataset contains 150 samples of iris flowers, categorized into three species (50 samples each): Setosa, Versicolor, and Virginica. It features four numeric attributes (measured in cm) used to predict the species: sepal length, sepal width, petal length, and petal width.

```
iris = load_iris()
iris_df = pd.DataFrame(iris.data,columns=iris.feature_names)
iris_df.head()
```

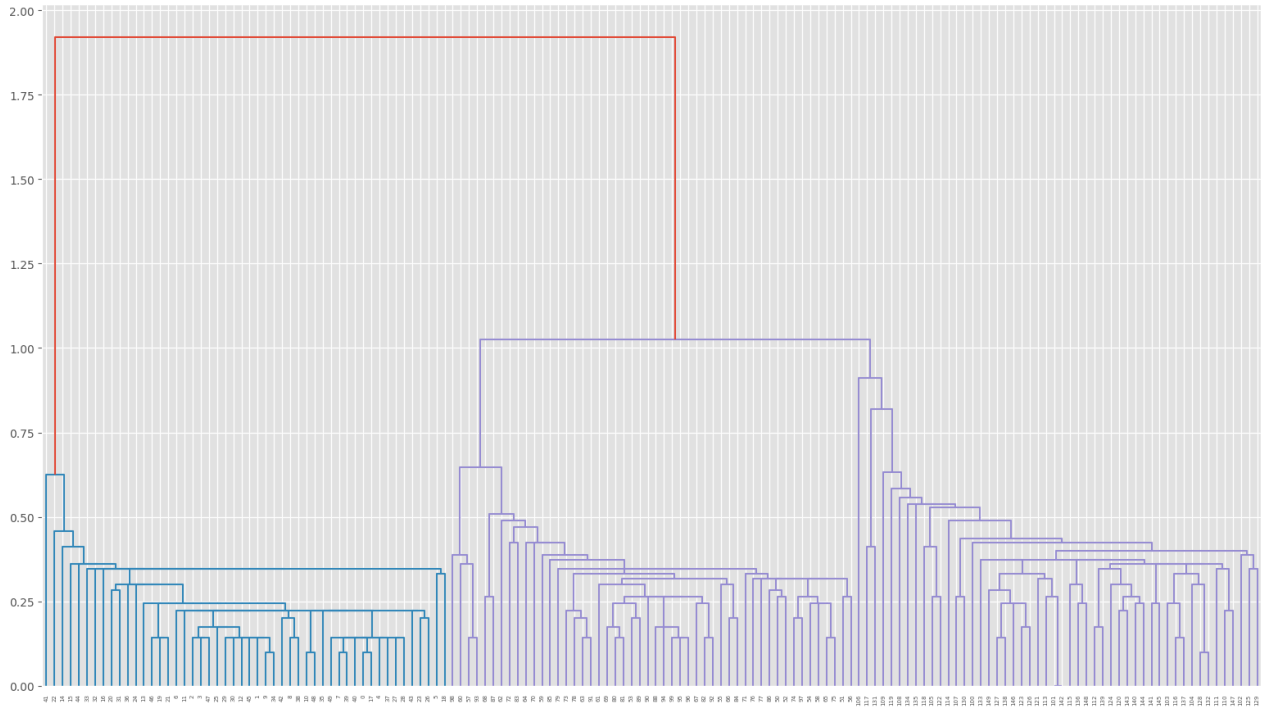
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
iris_df['class'] = iris.target
iris_df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Generating the distance metric with single linkage

```
distance_matrix = linkage(iris_df,method="single")
plt.style.use("ggplot")
plt.figure(figsize=(19.2,10.8))
dendrogram(distance_matrix)
plt.show()
```



DBScan

Moon Dataset

- Moon dataset is a simple toy dataset to visualize clustering and classification algorithms.

```
moon = make_moons(n_samples=500,noise=.05)
moon
```

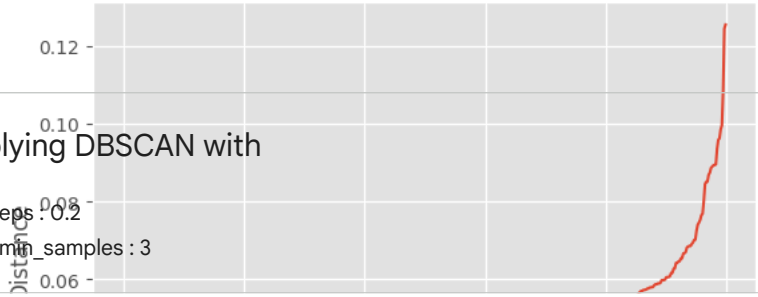
```
(array([[ 1.96149364e+00,  1.04553942e-01],
        [-6.45843694e-01,  7.53737563e-01],
        [ 1.76461203e+00, -1.85664319e-02],
        [ 6.89815692e-01,  8.55851227e-01],
        [ 1.01841596e+00,  1.14003059e-01],
        [-8.13669114e-01,  5.50324683e-01],
```

```
       [ 6.21777673e-01,  7.71456960e-01],
       [-2.26856434e-01,  1.02950747e+00],
       [ 1.03911753e+00,  1.33237605e-01],
       [ 1.46283841e+00, -4.15608581e-01],
       [-7.12553183e-01,  7.37282698e-01],
       [ 1.46279931e+00, -3.92616489e-01],
       [ 6.86280456e-01,  6.15408359e-01],
       [ 9.85584063e-01, -8.95884664e-03],
       [-5.60266076e-01,  8.08346312e-01],
       [ 3.98198310e-01, -3.75283383e-01],
       [ 1.38684708e+00, -4.32552671e-01],
       [ 4.58890233e-01,  9.22371892e-01],
       [ 1.11424304e-01,  7.12057561e-02],
       [-5.93804980e-01,  7.76942575e-01],
       [ 3.64147680e-01,  8.94280553e-01],
       [ 4.73795306e-01, -3.50430628e-01],
       [ 8.81702092e-01,  2.73389041e-01],
       [-8.22603991e-01,  4.19088022e-01],
       [ 3.97823338e-01,  9.29772017e-01],
       [-8.59434880e-01,  3.97641933e-01],
       [ 2.62945587e-01, -2.27136404e-01],
       [-3.92078500e-01,  9.22757913e-01],
       [ 1.23313363e+00, -5.04175068e-01],
       [-8.91377822e-01,  1.52490806e-01],
       [ 2.81638555e-01,  8.88283723e-01],
       [-6.10118063e-01,  7.82408365e-01],
       [ 6.40786817e-01,  7.49654839e-01],
       [ 6.12862098e-01,  8.65068223e-01],
       [ 1.00727875e+00,  1.93686777e-02],
       [ 1.51108752e+00, -4.13566863e-01],
       [-6.04356418e-03,  3.95899160e-01],
       [-6.12756376e-01,  8.45269163e-01],
       [ 2.70397384e-01, -1.39951311e-01],
       [ 8.39393546e-01,  6.46114075e-01],
       [ 4.94299923e-01,  8.74712750e-01],
       [-1.11112387e+00,  2.23777245e-01],
       [ 4.58983381e-01,  8.71873244e-01],
       [ 1.94879558e+00,  2.03055734e-01],
       [ 1.84153869e+00,  3.49455556e-02],
       [ 1.29607069e-01,  9.96136744e-01],
       [ 6.85552196e-01, -4.43705008e-01],
       [ 1.00435393e+00,  1.43680875e-01],
       [ 9.14042508e-01, -4.68203963e-01],
       [ 1.45938678e+00, -4.86785240e-01],
       [ 9.21541947e-01,  3.60098458e-01],
       [ 2.15680971e-01,  1.02289094e+00],
       [-9.23399046e-01,  9.26227308e-02],
       [ 6.36555451e-02,  2.05648980e-01],
       [ 6.36316235e-02,  3.23750909e-01],
       [ 1.36595354e+00, -4.49556669e-01],
       [ 1.48654153e+00, -4.21056064e-01],
```

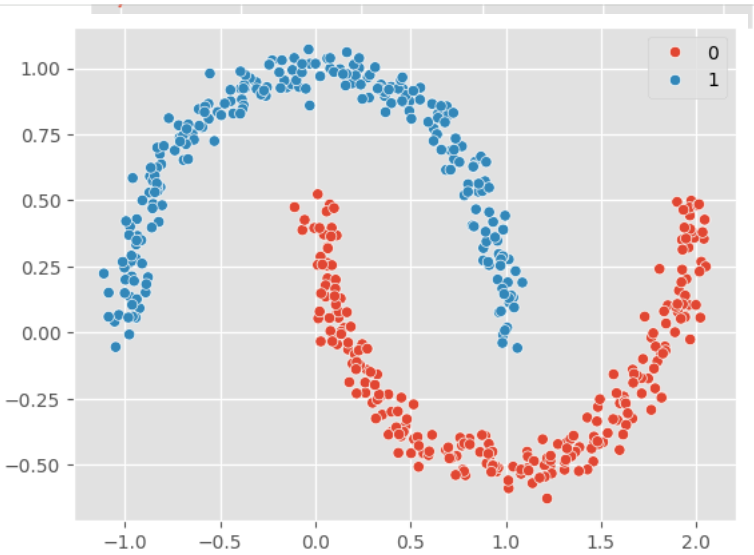
Determining optimal value of epsilon using elbow method

```
k = moon[0].shape[1] + 1 # k >= dimension of the dataset + 1
k_nearest_neighbour = NearestNeighbors(n_neighbors=k)
k_nearest_neighbour.fit(moon[0])
distances,indices = k_nearest_neighbour.kneighbors(moon[0])

distances = np.sort(distances[:,k-1])
plt.plot(distances)
plt.xlabel("Points")
plt.ylabel("Distance")
plt.show()
```



Applying DBSCAN with

- eps : 0.2
- min_samples : 3

```
db_scan = DBSCAN(eps=0.12,min_samples=k)
labels = db_scan.fit_predict(moon[0])

sns.scatterplot(x=moon[0][:,0],y=moon[0][:,1],hue=labels)
plt.show()
```